Gavin McClure-Coleman

CPSC450

DFS Sudoku Solver

Section 1 - Planning and Design:

Rules:

1. 9x9 board
2. Each row contains the digits 1 through 9 without repetition
3. Each column contains the digits 1 through 9 without repetition
4. Each of the 3x3 subgrids contains the digits 1 through 9 without repetition
5. The puzzle will start with some numbers already filled in, the goal is to fill in the remaining cells while satisfying the above rules

Approach:

The general approach I will be taking will be using Depth-First Search (DFS) recursion with backtracking:

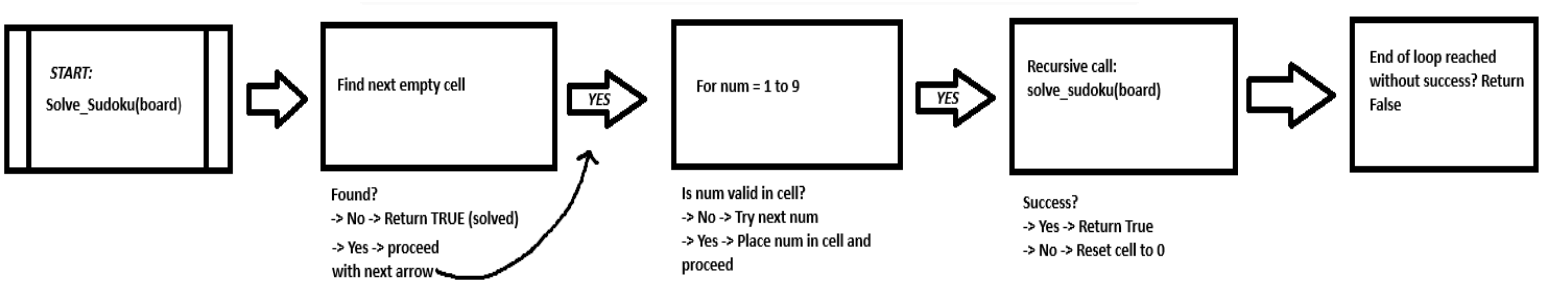
- First we search through the board by finding an empty cell, trying candidate numbers 1 through 9, and checking whether or not a candidate number violates any of sudokus constraints
- If a candidate choice is valid (doesn't violate any of the above rules), we can make a recursive call to continue solving the rest of the board in the same way
- If a dead end is reached (no possible numbers can be placed in the cell without breaking any rules), then we backtrack, undoing the last assignment and trying the next available candidate

Step-By-Step:

1. Find an empty cell (scan the grid to locate a cell that has not been filled yet (this will be indicated by a 0)
2. Termination check (if no empty cells remain, the puzzle is solved so we can return true)
3. Candidate iteration (loop through numbers 1-9)
4. Constraint checking (for each number, check if placing it in the current cell violates the row, column, or 3x3 box constraints
5. Recursive aspect (if valid, place the number in the cell. Recursively call the solve function for the updated board. If the recursive call reuters true, propagate the success back up)
6. Backtracking (assuming the recursive call has failed, reset the cell back to empty (0) and continue with the next candidate number before)

7. Exhaustion (if all numbers 1-9 are attempted and none lead to a valid solution, return false which triggers the backtracking to the parent cell)

Flowchart (this is pretty rough but give a decent idea):



Pseudocode:

```
def solve_sudoku(board):
        empty _cell = find_empty_cell(board)
        if not empty_cell do:
                return true
        row, col = empty_cell
        for num in range(1, 10):
                if is_valid_placement(board, num, (row, col)):
                        board[row][col] = num
                        if solve_sudoku(board):
                                return true
                        board[row][col] = 0
        return false
def is_valid_placement(board, num, pos):
        row, col = pos
        if num in board[row]:
                return false
        if num in [board[r][col] for r in range(9)]:
                return false
box_row_start = (row // 3) * 3
box_col_start = (col // 3) * 3
for r in range(box_row_start, box_row_start + 3):
        for c in range(box_col_start, box_col_start + 3):
                if board[r][c] == num:
                        return false
return true
```

Potential difficulties:

The main difficulty I see myself running into is time constraints. This solution to me seems really slow and I almost feel like DFS is not really the best way to solve this problem. It is possible that as I work on the code I will make some optimizations/adjustments to help with this but we will see. Other more obvious problems may also occur (testing errors, syntax errors, logic errors, etc.) as I write code as is pretty normal.

Section 2:

SEE MY CODE IN CPP/HPP FILES FOR THIS SECTION

Section 3 - Testing Plan:

My testing plan is to test that my solution meets the requirements of the core sudoku rules, as well as test that the output solutions for the test input boards are valid solutions. There will also be tests for things like unsolvable puzzles, board reading fails, and other edge cases. These can be found in the code and have been left out of this document to keep this section concise. Lastly, the make file automates the Post-Solve verification step of the 6 sudoku_<difficulty>.txt files.

Example unit tests:

- test_is_legal_value_row()
    - Fails when the same number is found more than once in the same row
    - Passes when all numbers in a row are unique digits 1-9
- test_is_legal_value_col()
    - Fails when the same number is found more than once in the same column
    - Passes when all numbers in a column are unique digits 1-9
- test_is_legal_value_box()
    - Fails when the same number is found more than once in the same box
    - Passes when all numbers in a 3x3 box are unique digits 1-9
- test_solve_simple()

- Fails if the solver fails to correctly solve a nearly complete board
- Passes if the solver correctly solves a nearly complete board

There are more unit tests that I will add to my code but to meet the 1 page limit I will have to omit from this section here. Hopefully these example tests give a good idea of what is being tested via C++ unit testing.

Section 4 - Reflection:

Overall, this project went pretty smoothly. I found the actual implementation of the DFS code for solving sudoku problems to be the quickest part. I found the most conflict with the original drafting/planning phase as well as some hiccups in testing. For starters, I feel as though DFS is not the most efficient or intuitive approach for solving sudoku, which made thinking of how to implement it at first a bit troublesome. Luckily, the pseudocode snippets in the class slides on DFS gave an outline for the code that I could base my approach around so that was helpful. As far as testing goes, I found thinking of good tests past the core main sudoku rule tests a bit tougher to think of. In that regard, if I were to do something differently going back I would have written a helper function much sooner that created a sample valid board so that I could run/make some more thorough tests, but I didn't do this until later in my process which led to me needing to manually copy over example boards from the sudoku.com website for testing which was quite tedious. All in all though the project went well for me I would say it performs pretty well at least for the 9x9 board it seems to run quickly on both my desktop and laptop. If anything this project has just made me more interested to see what some more optimal sudoku solver solutions do/what algorithms they incorporate (assuming DFS is not the most optimal).